

How to Use tncctest

John Langner, WB2OSZ

First Rough Draft, December 2025

Purpose:

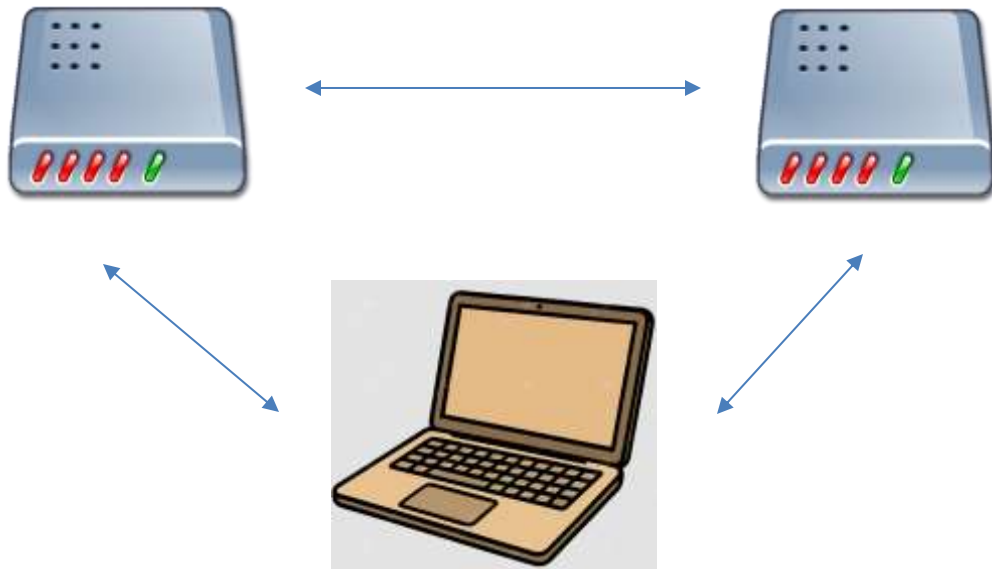
The Dire Wolf software TNC started out with only APRS in mind. Testing APRS is easy because there is a steady stream of packets surrounding us. All packets are independent and the receiving station doesn't need to accumulate state information.

Connected mode packet radio was added starting in 2016. This is vastly more difficult to test because it has to deal with so many different conditions, such as:

- Compatibility with AX.25 versions 2.0 and 2.2.
- Initiate and receive connection requests.
- Negotiation of various parameters such as maximum frame information size, window size, and retransmission strategies.
- Timeouts.
- Recovery for missed packets and other error conditions.
- Flow control when either TNC can't keep up.
- In theory we should be able to have many concurrent connections with different stations. (This would probably be most useful for a server which can handle multiple users at the same time.)

How can we test all of these conditions? Manually testing everything, over and over again, is not feasible so we need some sort of repeatable automated test to cover many of the various conditions.

The test setup is composed of three parts: two TNCs and the test fixture.



Usage

The application attaches to two TNCs, sends data back and forth between them, and verifies correct results. Artificial transmission errors are introduced, by TNC options, to test error detection and recovery.

The computer accesses a TNC by either:

- Traditional serial port, generally a USB cable nowadays.
- AGW protocol over TCP/IP for a software TNC.

The TNCs must be able to talk to each other by one of these methods:

- Wired audio connection. PTT is not needed.
- RF. Necessary if using radio with built-in TNC. e.g. TM-D710.
- [Virtual audio cable](#), for two software TNCs. Since we are running half duplex, we can use a single cable for both directions.

Example 1: Two Instances of direwolf, Send File

In this example, we send a text file of 243 lines and verify that number of lines sent and received are the same.

Software TNC Configurations:

dw0.conf	dw1.conf
adevice "CABLE Output" "CABLE Input" mycall DW0 kissport 0 agwport 9000 retry 20	adevice "CABLE Output" "CABLE Input" mycall DW1 kissport 0 agwport 9001 retry 20

The TNC addresses (mycall – station name) must be DW0 and DW1. It is hardcoded in the application.

Software TNC start up commands:

direwolf -c dw0.conf -q h -T %M:%S -E R20 -d s -d o -O dw0.log	direwolf -c dw1.conf -q h -T %M:%S -E R20 -d s -d o -O dw1.log
---	---

What to the command line options mean?

- `-c file` = configuration file.
- `-q h` = be quiet about distracting information.
- `-T spec` = timestamp format, just minutes and seconds here.
- `-E Rnn` = randomly discard specified % of **R**eceived frames.
- `-d o` = print change in status of PTT and DCD.
- `-d c` = print changes in state, variables, and timers.
- `-d cc` = also print even more detailed debug information.
- `-d s` = print statistics at the end.
- `-O file` = save all console output to a log file.

Run the test harness.

```
tnctest -f poems.txt 9000=dw0 9001=dw1
```

In this case, -f means send a file. Sending data in only one direction makes it easy to see what is going on.

The full example is rather long, so we will just look at an interesting part, from the receiving side, DW1, and explain what is going on.

Key:

- Blue = Received frames
- Magenta = Transmitted frames
- Black = Informational
- Green = Debug information. In this case “-d o” to show changes in Push to Talk (PTT) and Data Carrier Detect (DCD).

*We ask for missing frames 68, 71, 74, 76, 81, and 84.
If the other station does not understand SREJ-Multi (as determined by the XID negotiation) we would send six separate SREJ responses. The first would have f=1 and the others f=0.*

```
PTT 0 = 1
[0L 45:05] DW1>DW0:(SREJ res, N(R)=68, f=1) +71 +74 +76 +81 +84
PTT 0 = 0
DCD 0 = 1
```

Other station starts filling in missing frames but 71, 74, and 84 are lost. Additional frames 87-91 are received, limited by the window size.

```
[0.3 45:07] DW0>DW1:(I cmd, N(S)=68, N(R)=0, p=0, pid=0xf0)And welcomes
little fishes in,
Intentionally dropping incoming frame. Recv Error rate = 20 per cent.
Intentionally dropping incoming frame. Recv Error rate = 20 per cent.
```

```
[0.3 45:07] DW0>DW1:(I cmd, N(S)=76, N(R)=0, p=0, pid=0xf0) Did gyre and
gimble in the wabe;
```

```
[0.3 45:08] DW0>DW1:(I cmd, N(S)=81, N(R)=0, p=0, pid=0xf0) The jaws that
bite, the claws that catch!
Intentionally dropping incoming frame. Recv Error rate = 20 per cent.
```

```
[0.3 45:08] DW0>DW1:(I cmd, N(S)=87, N(R)=0, p=0, pid=0xf0)So rested he by
the Tumtum tree,
```

```
[0.3 45:09] DW0>DW1:(I cmd, N(S)=88, N(R)=0, p=0, pid=0xf0) And stood
awhile in thought.
```

```
[0.3 45:09] DW0>DW1:(I cmd, N(S)=89, N(R)=0, p=0, pid=0xf0)+
```

[0.3 45:09] DW0>DW1:(I cmd, N(S)=90, N(R)=0, p=0, pid=0xf0) And, as in
uffish thought he stood,

[0.3 45:10] DW0>DW1:(I cmd, N(S)=91, N(R)=0, p=0, pid=0xf0) The
Jabberwock, with eyes of flame,

DCD 0 = 0

PTT 0 = 1

*We ask for retransmission of 71, 74, and 84. F=1 means acknowledge
one less than the first one, i.e. 70.*

[0L 45:10] DW1>DW0:(SREJ res, N(R)=71, f=1) +74 +84

PTT 0 = 0

DCD 0 = 1

*The other station did not receive the SREJ, timed out, and asks what is
your current state? RR command with P=1 means an answer is
expected.*

[0.2 45:14] DW0>DW1:(RR cmd, N(R)=0, p=1)

DCD 0 = 0

PTT 0 = 1

We send the same thing again.

[0L 45:14] DW1>DW0:(SREJ res, N(R)=71, f=1) +74 +84

PTT 0 = 0

DCD 0 = 1

We receive repeats of all the missing frames.

[0.3 45:16] DW0>DW1:(I cmd, N(S)=71, N(R)=0, p=0,
pid=0xf0)=====

[0.3 45:16] DW0>DW1:(I cmd, N(S)=74, N(R)=0, p=0, pid=0xf0)+

[0.3 45:16] DW0>DW1:(I cmd, N(S)=84, N(R)=0, p=0, pid=0xf0)+

DCD 0 = 0

PTT 0 = 1

*Nothing is missing so we send RR rather than SREJ that we've been
seeing. This means up to 91 is acknowledged and the next expected is
92.*

[0L 45:16] DW1>DW0:(RR res, N(R)=92, f=0)

PTT 0 = 0

When finished, the statistics might look something like this:

0 I frames received	245 I frames received
24 RR frames received	41 RR frames received
0 RNR frames received	0 RNR frames received
0 REJ frames received	0 REJ frames received

42 SREJ frames received	0 SREJ frames received
0 SABME frames received	2 SABME frames received
0 SABM frames received	0 SABM frames received
0 DISC frames received	1 DISC frames received
0 DM frames received	0 DM frames received
2 UA frames received	0 UA frames received
0 FRMR frames received	0 FRMR frames received
0 UI frames received	0 UI frames received
1 XID frames received	1 XID frames received
0 TEST frames received	0 TEST frames received
5 peak retry count	0 peak retry count

Example 2: Two Instances of direwolf, Bidirectional

This is more interesting. And more complicated to figure out what happened if something goes wrong.

The sending side, DW0, sends a bunch of frames such as:

```
0001 send data
0002 send data
0003 send data
...
```

The other station, DW1, changes “send data” to “reply” and sends it in the other direction. e.g.

```
0001 reply
0002 reply
0003 reply
...
```

The test harness verifies everything comes back in the correct order.

Software TNC Configurations:

dw0.conf	dw1.conf
adevice "CABLE Output" "CABLE Input" mycall DW0 kissport 0 agwport 9000 retry 20 maxv22 10	adevice "CABLE Output" "CABLE Input" mycall DW1 kissport 0 agwport 9001 retry 20

Note the addition of MAXV22. With high error packet error rates, there were cases where it gave up on SABME and fell back to SABM. Note to self: Should probably also try this forcing SABM.

The TNC addresses (mycall – station name) must be DW0 and DW1 even it is some other kind of TNC. It is hardcoded in the application.

Software TNC start up commands:

direwolf -c dw0.conf -q h -T %M:%S -E R20 -d s -d o -O dw0.log	direwolf -c dw1.conf -q h -T %M:%S -E R20 -d o -O dw1.log
---	--

What do the command line options mean?

- `-c file` = configuration file.
- `-q h` = be quiet about distracting information.
- `-T spec` = timestamp format, just minutes and seconds here.
- `-E Rnn` = randomly discard specified % of Received frames.
- `-d o` = print change in status of PTT and DCD.
- `-d c` = print changes in state, variables, and timers.
- `-d cc` = also print even more detailed debug information.
- `-d s` = print statistics at the end.
- `-O file` = save all console output to a log file.

Now for a more interesting example.

```
tnctest -n 1000 9000=dw0 9001=dw1
```

In this case, `-n` specifies the length of the test. The default of 9999 runs about 3 ½ hours with 1200 bps.

The full example is rather long, so we will just look at an interesting part, from DW1, and explain what is going on.

Key:

- Blue = Received frames
- Magenta = Transmitted frames
- Black = Informational
- Green = Debug information. In this case “-d o” to show changes in Push to Talk (PTT) and Data Carrier Detect (DCD).

Here is something we did not see in the previous example.

```
[0.3 55:18] DW0>DW1:(I cmd, N(S)=64, N(R)=48, p=0, pid=0xf0)0705 send  
data<0x0d>
```

...

*(Lots of back and forth here for fill-ins. The important thing to note is
that the highest N(S) seen so far was 64.*

...

```
[0.3 55:48] DW0>DW1:(I cmd, N(S)=33, N(R)=59, p=0, pid=0xf0)0674 send  
data<0x0d>
```



```
[0.3 55:48] DW0>DW1:(I cmd, N(S)=53, N(R)=59, p=0, pid=0xf0)0694 send data<0x0d>
```

```
[0.3 55:48] DW0>DW1:(I cmd, N(S)=56, N(R)=59, p=0, pid=0xf0)0697 send data<0x0d>
```

```
[0.3 55:49] DW0>DW1:(I cmd, N(S)=59, N(R)=59, p=0, pid=0xf0)0700 send data<0x0d>
```

```
DCD 0 = 0
```

```
PTT 0 = 1
```

In the previous one-way test, we used RR to acknowledge the last contiguous sequence number and say we are ready for the next one.

In the two-way test, the I frame can provide the acknowledgement. After receiving an I frame with sequence N(S)=59, we are all caught with everything through 64. Here we include the N(R)=65 acknowledgement in the I frame and don't need to send an RR.

```
[OL 55:50] DW1>DW0:(I cmd, N(S)=59, N(R)=65, p=0, pid=0xf0)0674 reply<0x0d>
```

```
[OL 55:50] DW1>DW0:(I cmd, N(S)=60, N(R)=65, p=0, pid=0xf0)0675 reply<0x0d>
```

```
[OL 55:50] DW1>DW0:(I cmd, N(S)=61, N(R)=65, p=0, pid=0xf0)0676 reply<0x0d>
```

```
[OL 55:50] DW1>DW0:(I cmd, N(S)=62, N(R)=65, p=0, pid=0xf0)0677 reply<0x0d>
```

```
[OL 55:50] DW1>DW0:(I cmd, N(S)=63, N(R)=65, p=0, pid=0xf0)0678 reply<0x0d>
```

```
[OL 55:50] DW1>DW0:(I cmd, N(S)=64, N(R)=65, p=0, pid=0xf0)0679 reply<0x0d>
```

At the end of the test, tnctest displays:

```
[R 11856.853] 9997 reply
```

```
[R 11856.853] 9998 reply
```

```
[R 11856.854] 9999 reply
```

```
-----  
SUCCESS! Got last expected reply.  
-----
```

```
[T 11857.847] *** Send disconnect request ***
```

```
[R 11858.557] *** Disconnected from DW0 ***
```

```
[R 11859.577] *** Disconnected from DW1 ***
```

```
Success!
```

The numbers are elapsed time, from the start, in seconds. In this case it ran for about 3 ½ hours.

Statistics area displayed by the TNCs after the link is disconnected.

10118 I frames received 653 RR frames received 0 RNR frames received 0 REJ frames received	10105 I frames received 690 RR frames received 0 RNR frames received 0 REJ frames received
---	---

594 SREJ frames received	622 SREJ frames received
0 SABME frames received	2 SABME frames received
0 SABM frames received	0 SABM frames received
0 DISC frames received	1 DISC frames received
0 DM frames received	0 DM frames received
2 UA frames received	0 UA frames received
0 FRMR frames received	0 FRMR frames received
0 UI frames received	0 UI frames received
1 XID frames received	3 XID frames received
0 TEST frames received	0 TEST frames received
9 peak retry count	10 peak retry count

Example 3: direwolf & OpenTNC, Send File

This is an interoperability test.

- [Direwolf](#) is a software TNC that runs on Windows, Mac OSX, Linux, OpenBSD, etc.
- [OpenTNC](#) is low-cost modern hardware based TNC reminiscent of the TAPR TNC-2.

They are independently developed open-source implementations of AX.25 v2.2. Open-source means that you can see how they work and make enhancements. Open-source projects can continue to thrive after the authors go SK or lose interest.

They both include [FX.25](#) forward error correction and additional modem types and speeds besides the ubiquitous 1200 bps. FX.25 is not enabled for this test.

Let's see if they play well together.

In this case direwolf will talk to a USB audio device which is connected to audio connector on the Open TNC. Conveniently, it is the same pin out as the Kantronics KPC-3+ so the same cables can be reused.

In my case, the OpenTNC shows up as COM6 in the device manager.

Software versions:

- Direwolf 1.9 development D (dev branch, July 2026)
- OpenTNC 6.2.3a1

Configurations:

dw0-usb.conf – for direwolf	Configuration typed into OpenTNC
adevice "USB" mycall DW0 kissport 0 agwport 9000 retry 20	Not needed because tnctest issues “reset” command and sets required parameters.

Software TNC start up commands:

direwolf -c dw0-usb.conf -q h -T %M:%S -E 20 -E R20 -d s -O dw0.log	(none)
--	--------

Note that we are introducing errors in both directions.

- -E 20 Corrupts CRC of 20% of transmitted frames.
- -E R20 Drops 20% of Received frames.

Run the test harness.

```
tnctest -f poems.txt 9000=dw0 COM7=OpenTNC
```

Here are a few interesting parts.

The link is established on the second SABME sent.

Attempting connect to TNC1 ...

[0L 27:36] DW0>TNC1:(SABME cmd, p=1)

Intentionally sending invalid CRC for frame above. Xmit Error rate = 20 per cent.

[0L 27:41] DW0>TNC1:(SABME cmd, p=1)

[0.2 27:44] TNC1>DW0:(UA res, f=1)

Stream 4: Connected to TNC1. (v2.2)

In the XID negotiation, direwolf would like to use a transmit window size of 32. OpenTNC says it can only handle 16. The transmit window size (EMAXFRAME) parameter is reduced to 16 for this session.

[0L 27:45] DW0>TNC1:(XID cmd, p=1) Half-Duplex REJ SREJ Multi-SREJ modulo-128 I-Field-Length-Tx=256 I-Field-Length-Rx=2048 window-Size-Tx=32 window-Size-Rx=63 Ack-Timer=4000 Retries=20

[0.2 27:47] TNC1>DW0:(XID res, f=1) Half-Duplex SREJ modulo-128 I-Field-Length-Tx=256 I-Field-Length-Rx=512 Window-Size-Tx=2 Window-Size-Rx=16 Ack-Timer=6000 Retries=20

Reducing our EMAXFRAME from 32 to 16 as result of XID exchange.

Data transfer begins. Two of the information frames get clobbered.

[0L 27:50] DW0>TNC1:(I cmd, N(S)=0, N(R)=0, p=0, pid=0xf0)ZCZC<0x0d>

Intentionally sending invalid CRC for frame above. Xmit Error rate = 20 per cent.

[0L 27:50] DW0>TNC1:(I cmd, N(S)=1, N(R)=0, p=0, pid=0xf0)Poems by<0x0d>

[0L 27:50] DW0>TNC1:(I cmd, N(S)=2, N(R)=0, p=0, pid=0xf0)Lewis

Carroll<0x0d>

[0L 27:50] DW0>TNC1:(I cmd, N(S)=3, N(R)=0, p=0, pid=0xf0)1832 - 1898<0x0d>

[0L 27:50] DW0>TNC1:(I cmd, N(S)=4, N(R)=0, p=0, pid=0xf0)<0x0d>

[0L 27:50] DW0>TNC1:(I cmd, N(S)=5, N(R)=0, p=0,

pid=0xf0)=====<0x0d>

[0L 27:50] DW0>TNC1:(I cmd, N(S)=6, N(R)=0, p=0, pid=0xf0)<0x0d>

[0L 27:50] DW0>TNC1:(I cmd, N(S)=7, N(R)=0, p=0, pid=0xf0)Acrostic <0x0d>

[0L 27:50] DW0>TNC1:(I cmd, N(S)=8, N(R)=0, p=0, pid=0xf0)<0x0d>

[0L 27:50] DW0>TNC1:(I cmd, N(S)=9, N(R)=0, p=0, pid=0xf0)Little maidens,
when you look<0x0d>

Intentionally sending invalid CRC for frame above. Xmit Error rate = 20 per cent.

[0L 27:51] DW0>TNC1:(I cmd, N(S)=10, N(R)=0, p=0, pid=0xf0)On this little story-book,<0x0d>

[0L 27:51] DW0>TNC1:(I cmd, N(S)=11, N(R)=0, p=0, pid=0xf0)Reading with attentive eye<0x0d>

[0L 27:51] DW0>TNC1:(I cmd, N(S)=12, N(R)=0, p=0, pid=0xf0)Its enticing history,<0x0d>

[0L 27:52] DW0>TNC1:(I cmd, N(S)=13, N(R)=0, p=0, pid=0xf0)Never think that hours of play<0x0d>

[0L 27:52] DW0>TNC1:(I cmd, N(S)=14, N(R)=0, p=0, pid=0xf0)Are your only HOLIDAY,<0x0d>

[0L 27:52] DW0>TNC1:(I cmd, N(S)=15, N(R)=0, p=0, pid=0xf0)And that in a HOUSE of joy<0x0d>

Notice that the transmit window is 16 rather than the normal default of 32. An SREJ is received for each of the missing I frames.

However we did not receive the srej for 9.

[0.3 27:57] TNC1>DW0:(SREJ res, N(R)=0, f=1)

Intentionally dropping incoming frame. Recv Error rate = 20 per cent.

[0L 27:57] DW0>TNC1:(I cmd, N(S)=0, N(R)=0, p=0, pid=0xf0)ZCZC<0x0d>

The srej 9 is sent to us again. We fill in 9 and continue with 16, etc.

[0.2 27:59] TNC1>DW0:(SREJ res, N(R)=9, f=1)

[0L 28:00] DW0>TNC1:(I cmd, N(S)=9, N(R)=0, p=0, pid=0xf0)Little maidens, when you look<0x0d>

[0L 28:00] DW0>TNC1:(I cmd, N(S)=16, N(R)=0, p=0, pid=0xf0)Lessons serve but to annoy:<0x0d>

Etc. etc. All goes well.

[0L 33:16] DW0>TNC1:(I cmd, N(S)=111, N(R)=0, p=0, pid=0xf0) They'd eaten every one.<0x0d>

[0L 33:16] DW0>TNC1:(I cmd, N(S)=112, N(R)=0, p=0, pid=0xf0)<0x0d>

Disconnect from TNC1 ...

[0L 33:18] DW0>TNC1:(DISC cmd, p=1)

[0.3 33:20] TNC1>DW0:(RR cmd, N(R)=116, p=0)

Intentionally dropping incoming frame. Recv Error rate = 20 per cent.

[0L 33:22] DW0>TNC1:(DISC cmd, p=1)

[0L 33:24] DW0>TNC1:(DISC cmd, p=1)

Intentionally dropping incoming frame. Recv Error rate = 20 per cent.

[0.2 33:27] TNC1>DW0:(DM res, f=1)

Stream 4: Disconnected from TNC1.

Finally, the statistics.

0 I frames received
10 RR frames received
0 RNR frames received
0 REJ frames received
65 SREJ frames received
0 SABME frames received
0 SABM frames received
0 DISC frames received

```
1 DM frames received
1 UA frames received
0 FRMR frames received
0 UI frames received
1 XID frames received
0 TEST frames received
10 peak retry count
```

At the end of test, tnc test displays:

```
because\r
[R 409.246] But answer came there none-\r
[R 409.256] And this was scarcely odd,

[R 409.256] They'd eaten every one.\r
[R 409.256] \r
[R 409.277] =====\r
[R 409.277] \r
[R 409.286] <EOF>\r
[R 409.286] NNNN\r

-----
SUCCESS! Got last expected reply.
-----
[T 410.054] *** Send disconnect request ***
[R 411.229] *** DISCONNECTED\r
DEBUG: is_connected[1] = 0

[R 411.240] \r
[R 411.240] \r
[R 411.251] cmd:

TNC 0 received 'K' data, data_len = 16
TNC 0 received 'd' data, data_len = 36
[R 412.384] *** Disconnected from TNC1 ***
Success!
```

Now, let's try something a little more interesting.

Example 4: direwolf & OpenTNC, Bidirectional

Configurations:

dw0-usb.conf – for direwolf	Configuration typed into OpenTNC
<pre>adevice "USB" mycall DW0 kissport 0 agwport 9000 retry 20 emaxframe 3</pre>	Not needed because tnctest issues “reset” command and sets required parameters.

Software TNC start up commands:

<pre>direwolf -c dw0-usb.conf -q h -T %M:%S -E 20 -E R20 -d s -E 20 -d s -O dw0.log</pre>	(none)
---	--------

Start the test harness:

```
tnctest -n 1000 9000=dw0 COM7=OpenTNC
```

To be continued...